

Execution-Grounded Automated AI Research: Teaching LLMs to Generate Ideas That Actually Work

The constructive sequel to the Ideation–Execution Gap. If proposal-stage novelty scores are a hackable proxy that inverts once ideas are built, the fix is obvious in principle and brutal in practice: **stop scoring ideas by how they sound and score them by what happens when you run them**. This paper builds the machinery to do exactly that at scale — an automated executor that turns a natural-language research idea into a real GPU experiment and returns its benchmark number as a reward — then asks whether an LLM can *learn* from that reward. Two learners are tried on two research problems you have personally run: the nanoGPT GPT-2 124M speedrun (pre-training) and GRPO math finetuning (post-training). The headline is a split decision: **evolutionary search discovers recipes that beat the baselines (69.4% vs 48.0% post-training; 19.7 vs 35.9 min pre-training) within ten epochs, while RL from execution reward raises the average idea quality but collapses in diversity and never lifts the maximum** — the one metric discovery actually cares about.

Background

Primary paper: Towards Execution-Grounded Automated AI Research (Chenglei Si, Zitong Yang, Yejin Choi, Emmanuel Candès, Diyi Yang, Tatsunori Hashimoto — Stanford, submitted 20 Jan 2026, arXiv 2601.14525). Open-source environments and idea-execution trajectories: github.com/NoviScl/Automated-AI-Researcher.

This is the third paper in a tight arc by the same Stanford group (SALT / Tatsu Lab), and it only makes sense as the *answer* to the first two:

1. **Can LLMs Generate Novel Research Ideas?** (Si, Yang, Hashimoto — Sep 2024, arXiv 2409.04109, ICLR 2025). The setup. 79 reviewers blindly judged LLM ideas **more novel than expert ideas (5.64 vs 4.84, $p < 0.01$)** — but only comparable-or-lower on feasibility. See the discussion in [ideation-execution-gap-guide.md](#).
2. **The Ideation–Execution Gap** (Si, Hashimoto, Yang — Jun 2025, arXiv 2506.20803). The negative result that motivates *this* paper. When 43 experts spent 100+ hours each actually building those ideas, the AI novelty edge **reversed** — the ideation score turned out to be nearly uncorrelated (and for excitement, *anti*-correlated) with executed value. Full treatment in [ideation-execution-gap-guide.md](#). The one-line diagnosis: idea-stage review is a reward model, and the idea generators have reward-hacked it.
3. **Towards Execution-Grounded Automated AI Research** (this paper, Jan 2026). The constructive fix. If the proxy is invalid, replace it with the target: execute the idea and use the *real* benchmark result as the training signal. The paper's whole thesis is one sentence from the abstract — "*Execution grounding may help, but it is unclear whether automated execution is feasible and whether LLMs can learn from the execution feedback.*" This paper settles the feasibility question (yes) and gives a nuanced, mostly-cautionary answer to the learning question.

Two more lineages feed the method, not the motivation:

1. **The modded-nanoGPT speedrun** (github.com/KellerJordan/modded-nanogpt, Jordan et al. 2024) is the pre-training environment's substrate — the community competition to train GPT-2 124M to 3.28 FineWeb val loss on 8×H100 as fast as possible (45 min in Jun 2024 → **under 2.1 min by Dec 2025**). The related Automated LLM Speedrunning Benchmark (Meta, Jun 2025) measures whether agents can *reproduce* known record transitions — a sibling problem discussed in the "vs alternatives" table below and in the forthcoming [llm-speedrunning-benchmark-guide.md](#).

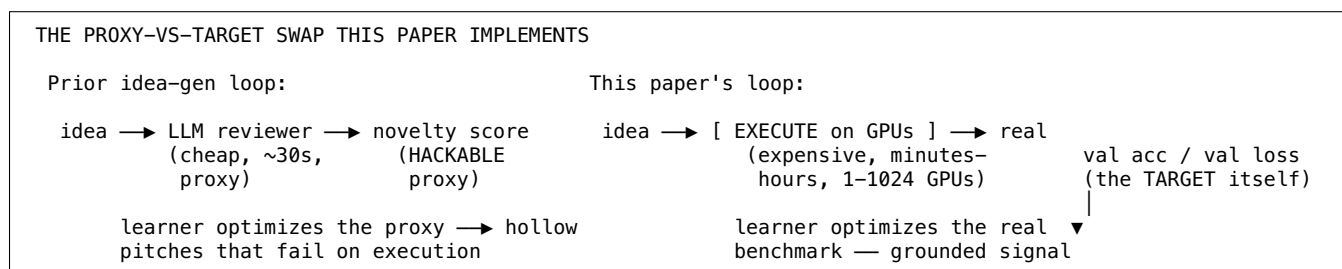
2. **AlphaEvolve** (arXiv 2506.13131, Novikov et al., DeepMind, May 2025) is the direct inspiration for the search scaffold — an evolutionary coding agent that edits code and selects on a programmatic evaluator. This paper lifts that pattern up one level of abstraction, from *code diffs* to *natural-language research ideas*, and adds an RL variant AlphaEvolve does not have.

Situate this against the wider field in `landscape-2025-2026.md` (execution grounding is Open Problem #1 there) and the forthcoming `iclr-2026-rsi-workshop.md` on recursive self-improvement.

What Problem Does This Solve?

Every "AI scientist" pipeline in the landscape guide — AI Scientist, Robin, AI Co-Scientist, data-to-paper — generates ideas and scores them with an LLM reviewer, a BTL/Elo ranker, or a small human panel. The Ideation–Execution Gap proved that score is an invalid proxy: *novel-sounding* ideas systematically lose to trivial baselines once run. So the ranking signal the whole genre optimizes is the wrong signal.

The obvious repair — "just execute the ideas and rank by the real result" — is exactly what nobody had done at scale, because executing an idea is expensive, fiddly, and open-ended. You need something that can take *any* natural-language idea ("mix a learned local-context compression into each attention layer", "drop GRPO's importance-weighting and clip") and, without a human in the loop, turn it into working code, run it on the right hardware under a controlled budget, and hand back a trustworthy number. That artifact is the paper's first contribution: **a high-throughput automated idea executor** that acts as a reward function *idea* → *real benchmark score*. Its second contribution is to plug that reward into two learning algorithms and *characterize what happens* — which turns out to be the interesting part.



Crucially, the target is **open-ended**: unlike math RLVR, there is no gold "correct idea." The reward is the very quantity you are trying to maximize (does this idea make the model better?), so a high reward is by construction a genuinely good research idea, not a match to a known answer.

Key Terms

Execution-grounded reward — the reward assigned to an idea *by running it*. The automated executor implements the idea as a code diff against a baseline codebase, launches the resulting training job on GPUs under a fixed budget, and maps the measured benchmark to a scalar: validation accuracy (post-training) or $1/\text{val_loss}$ (pre-training). A failed execution (code bugs, unpatchable diff, crash) gets reward **0**. This is not a learned reward model and not a rule-check against a known answer — it is the empirical outcome of a real experiment.

Ideator vs executor — the paper cleanly separates the LLM being *taught* (the **ideator**, which emits natural-language ideas) from the LLM/machinery that *implements and runs* them (the **executor**). In both learners, **only the ideator is updated**; the executor is a fixed reward function. "Self-execution" means the same model plays both roles.

Execution-guided evolutionary search — a gradient-free optimizer (roots in genetic programming, Koza 1994) that maintains a population of ideas across epochs and, each epoch, splits generation into **exploitation** (recombine ideas that *beat the baseline* — append them to the prompt and ask the ideator to combine their

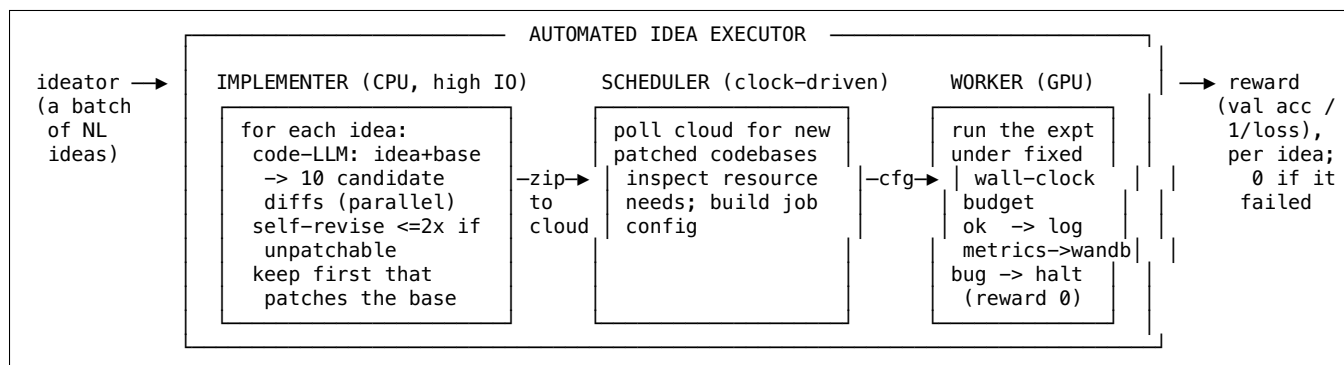
strengths) and **exploration** (sample old ideas and ask for something *different*). It is a **quality-diversity** method: it deliberately keeps a diverse frontier so it can find the rare high-value idea, not just the average one. The ideator's weights are never touched — the "learning" is in-context, via prompt evolution.

RL from execution reward — standard GRPO where the reward is the execution-grounded reward above, applied to a single fixed prompt (the research environment). The ideator's *weights* are updated by policy gradient. This is the variant to contrast against the GRPO you studied (see the dedicated section).

Mode / diversity collapse — the failure where an RL policy converges onto a few high-reward, easy-to-produce outputs and loses the variety needed to ever find something better. Here it manifests as the ideator converging on ~2 trivial ideas and shrinking its thinking traces. It is the open-ended analogue of pass@k stagnation in RLVR (Yue et al. 2025; Wu et al. "The Invisible Leash", 2507.14843).

The Automated Idea Executor

This is the engineering contribution and the thing that makes the reward possible. It is a three-stage pipeline (Figure 1) exposed as a high-level API: **batch of natural-language ideas in** → **benchmark score per idea out**.



- **Implementer** (CPU, high IO): for each idea, makes parallel calls to a code-execution LLM to produce a diff against the baseline codebase. It samples **10 candidate diffs**; if a diff won't patch cleanly it feeds the patch error back and lets the model self-revise **up to 2 times**; the first diff that applies wins, and the patched codebase is zipped to a cloud bucket. (Prompting with both the idea *and* the baseline improves diff quality.)
- **Scheduler** (clock-driven middle layer): under a set clock frequency, downloads new codebases, examines the GPU resource requirement, and prepares a job config.
- **Worker** (GPU cluster): runs the experiment under a **fixed wall-clock budget**; on success uploads all metrics + full metadata (idea, code diff, execution log) to a bucket; on failure (code bugs) it halts and the idea scores 0. The ideator later downloads the batch of results.

As a reward function the whole thing reads:

```

def execution_reward(idea, env):          # env = {baseline_codebase, benchmark, gpu_budget}
# --- Implementer (parallel, CPU) ---
    patched = None
    for k in range(10):                  # sample up to 10 candidate diffs
        diff = code_llm(idea, env.baseline) # NL idea -> code diff
        for _ in range(2):                # self-revise if unpatchable
            if applies(env.baseline, diff):
                patched = patch(env.baseline, diff); break
            diff = code_llm(idea, env.baseline, patch_error=last_error)
        if patched: break
    if patched is None:
        return 0.0                        # could not implement -> failed execution
    upload(zip(patched))                  # -> cloud bucket

# --- Scheduler + Worker (GPU) ---
    cfg = plan_resources(patched)
    try:
        metrics = run_on_gpu(patched, cfg, budget=env.gpu_budget) # a real training run
        return metrics.val_accuracy if env.is_posttrain \
            else 1.0 / metrics.val_loss # 1/loss for pre-train
    except (CodeBug, Crash, Timeout):
        return 0.0                        # failed execution -> zero reward

```

The **feasibility prerequisite** — that current LLMs can serve as both ideator and executor well enough to get a usable signal — is verified before any learning (Section 3): sampling and self-executing 50 ideas, **Claude-4.5-Opus and Claude-4.5-Sonnet execute >90%** of their own ideas successfully on nanoGPT, and even best-of-50 *without any search* already beats the baselines (Sonnet 60.4% max acc vs 48.0% on GRPO; Opus 3.237 min loss vs 3.255 on nanoGPT). With GPT-5 as a shared executor for different ideators, execution rate ranges 42–78%, and open-weight ideators (Kimi-K2-Thinking, Qwen3-235B-A22B) also clear the baselines. The loop is feasible; the question is whether an ideator can *learn* to do better than best-of-N.

The Two Research Environments (you have run both)

The paper deliberately picks two GPU-intensive problems that are open-ended (room for real algorithmic innovation) yet have well-established baselines and clean metrics — and they happen to be the two training regimes you have hand-implemented.

	PRE-TRAINING (nanoGPT)	POST-TRAINING (GRPO)
baseline code model	modded-nanoGPT speedrun (Jordan '24) 124M GPT-2 (Radford '19)	a GRPO impl (Shao et al. '24) Qwen2.5-Math-1.5B
data / benchmark	FineWeb; reach 3.28 val loss	MATH (Hendrycks '21); val acc
baseline result	35.9 min to reach 3.28 (loss 3.255 under the fixed 25-min search budget)	48.0% accuracy
search reward	1 / val_loss under a fixed 25-min training budget (avoids runtime variance; optimize loss, not time)	val accuracy under a fixed training-time budget
maps to YOUR	your GPT-2 124M nanoGPT run	your GRPO tutorial

Two anti-reward-hacking design choices are worth noting because they are exactly the traps an execution reward invites:

- **nanoGPT — attention leakage.** Early on, models changed the attention mechanism in ways that let a token peek at future tokens (trivially lowering loss). The fix: a frozen inference function that predicts **one token at a time** during the final validation, used for every run, so no future-token leak can inflate the score.
- **GRPO — validation tampering.** All evaluation hyperparameters are frozen and all validation-related code lives in a **separate file the executor cannot access or modify**, so the ideator can't "improve" accuracy by editing the metric.

Learner 1 — Execution-Guided Evolutionary Search

The scaffold (Algorithm 1) is an exploration/exploitation split with an annealing schedule, inspired by AlphaEvolve:

```

Algorithm 1 Execution-Guided Search (batch N, epochs T, baseline  $\beta$ )
I0 ← SampleIdeas(N); D0 ← execute(I0)           # {(idea, reward)}
for t = 1..T:
  a ← a(t)                                       # exploitation %, annealed up over t
  D+ ← { (idea,r) in D_{<t} : r >  $\beta$  }          # trajectories that BEAT the baseline
  N_exp ← floor(a/100 * N); N_expl ← N - N_exp
  I_exp ← ExploitVariants(D+, N_exp)            # recombine winners: "combine their strengths"
  I_expl ← ExploreNovel(subsample(D_{<t}), N_expl) # "generate something different from these"
  D_t ← execute(I_exp ∪ I_expl)
return union of all D_t

```

Start at **50% exploit / 50% explore** in epoch 1 and anneal toward exploitation later. Batch $N = 50$ (GRPO) / **80** (nanoGPT), run for **10 epochs**, self-execution with three frontier models (Claude-4.5-Opus, Claude-4.5-Sonnet, GPT-5). The key empirical claim: **search significantly beats best-of-N under the same sampling budget** — with $N=80/\text{epoch}$, best-of-N stays flat while search climbs from epoch 1 onward (Figure 4), proving the ideator genuinely leverages prior trajectories rather than just sampling more.

Numerical walkthrough — post-training, 48.0% → 69.4%

- Baseline GRPO on Qwen2.5-Math-1.5B / MATH: **48.0%**.
- Running the search with Claude-4.5-Sonnet as ideator+executor, the best solution appears at **epoch 2**: it finds that **vanilla policy gradient with a group-average baseline, *without* importance reweighting or clipping**, plus precise hyper-parameter tuning, outperforms the standard GRPO objective in this setup — and exploits that finding for the rest of the run. Result: **69.4%**, i.e. **+21.4 points** over baseline.
- For calibration, the best human solution on the Stanford CS336 leaderboard (grad students optimizing the *same* environment as a class assignment) is **68.8%** — the automated search edges it out.

There is a delicious lesson here for anyone who studied GRPO: the winning "idea" is essentially an *ablation of GRPO back toward REINFORCE-with-group-baseline*. At 1.5B / MATH scale under a fixed budget, GRPO's clipping and importance-sampling machinery was not load-bearing — dropping it and tuning helped. The executor discovered that empirically, by running it.

Numerical walkthrough — pre-training, 35.9 → 19.7 min

- Baseline modded-nanoGPT codebase reaches the 3.28 target in **35.9 min** on $8 \times H100$.
- Search with Claude-4.5-Opus (the only model showing a clear *scaling* trend across epochs) reaches min val loss **3.1407 at epoch 9** by stacking: wider SwiGLU ($5 \times$) with output scaling, learnable skip connections every 4 and 8 layers, separate attention/MLP residual scales, higher LR (0.00168), reduced weight decay (0.065), warmup 173, cosine, untied embeddings, $\beta_2=0.99$, and **EMA of intermediate checkpoints** at validation (full recipe in the paper's Appendix A.2).
- That solution, re-run under the *original* speedrun setup (train until 3.28), reaches the target in **19.7 min** — a **$\sim 1.82 \times$ speedup** over the 35.9-min baseline codebase.

nanoGPT wall-clock to 3.28 val loss ($8 \times H100$), log-ish scale:

baseline codebase		35.9 min	
Opus search soln		19.7 min	(1.82x faster)
best HUMAN speedrun		2.1 min	(Dec 2025 record)

beats the BASELINE it started from — but $\sim 9.4 \times$ slower than human SOTA

The final row is the honest one and the whole "reproduction vs discovery" caveat in miniature: the search improves on the *starting codebase* it was handed, but it is nowhere near the best human speedrun. It found real gains; it did not find frontier ones.

What kind of ideas? (hyper-parameter vs algorithmic)

An LLM-judge classifies every generated idea as hyper-parameter tuning vs algorithmic (Table 2, N=500 GRPO / 800 nanoGPT ideas). All models generate a substantial fraction of genuinely *algorithmic* ideas — this is not just knob-twiddling:

Model	— GRPO (acc, higher=better) —				— nanoGPT (loss, lower=better) —				
	metric	HP%	HP-best	Algo%	Algo-best	HP%	HP-best	Algo%	Algo-best
GPT-5		5.0%	50.2%	95.0%	60.0%	15.4%	3.195	84.6%	3.170
Claude-4.5-Sonnet		41.1%	69.4%	58.9%	67.4%	31.3%	3.208	68.7%	3.208
Claude-4.5-Opus		3.7%	50.4%	96.3%	61.6%	8.7%	3.147	91.3%	3.141

(Sonnet is the exception that tunes HP heavily; its best GRPO idea was HP-based. Everyone else's best ideas are algorithmic. Opus is the most algorithm-heavy and the best on nanoGPT.)

The paper also notes several ideas that *rediscover recent papers* — e.g. a proposed "response diversity reward" resembling Li et al. 2025, and a "causal context compression" resembling Allen-Zhu 2025's canon layers. Encouraging for feasibility, but a flag that this is **rediscovery within the recent literature**, not novel discovery (see Failure Modes).

Learner 2 — RL from Execution Reward (contrast to your GRPO)

This is the same GRPO algorithm you studied, wired to a very different reward. Base ideator: **Qwen3-30B-A3B**, standard GRPO (Shao et al. 2024), trained via the Tinker API. Here is the exact mapping and where it diverges from the math-RLVR GRPO in your tutorial:

	YOUR GRPO (math RLVR)	RL FROM EXECUTION REWARD (this paper)
prompt	a math problem (many prompts)	ONE fixed prompt = the research env (baseline codebase + "improve this"); one-prompt RLVR, cf. Wang et al. 2025
rollout / response	a CoT solution to the problem	a thinking trace + a natural-language research IDEA (max 8192 tokens; only the extracted idea, not the trace, is executed)
group size G	modest (e.g. 8-16)	256 (post-train) / 128 (pre-train)
reward	rule check vs the KNOWN gold answer (~free string match, deterministic)	EXECUTE the idea as a real GPU experiment; reward = val acc / (1/val loss); 0 if it fails to run
ground truth	known (the answer exists)	UNKNOWN — the reward IS the thing you're trying to discover (open-ended)
cost per reward	one forward pass / string compare	a full training run; G rollouts => 256 GPUs (GRPO env) or 1024 GPUs (nanoGPT env) in parallel per step
what "correct" means	matching a label	a genuinely better research idea

The conceptual point — **why execution reward is not a preference or verifier reward**. In your DPO work the reward is a learned r_ϕ over human preferences (a hackable proxy). In your GRPO tutorial the reward is a *verifier* over a known answer (trustworthy, but only defined because the answer exists). Execution-grounded reward is a third thing: it is verifiable (you actually ran it) *and* it targets an unknown (there is no gold "best idea"), so it can in principle drive genuine discovery rather than reproduction of a label. The cost is that computing a single reward is a distributed GPU training job — this is RL where one reward evaluation is a small research experiment.

The findings: average up, maximum flat, diversity gone

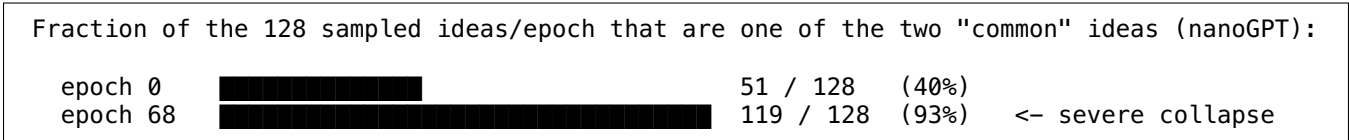
Positive — average reward increases (Figure 5, top row). For the first time in an *open-ended* research environment, RL lifts the mean idea quality with enough epochs:

- GRPO env: average accuracy **0.253** → **0.343** after 40 epochs.
- nanoGPT env: average reward **0.194** → **0.246** after 68 epochs (average val loss **5.150** → **4.066**).

This mirrors one-shot RLVR results on math — reassuring that the loop trains at all.

Negative — the maximum reward does not improve (Figure 5, bottom row): it *fluctuates with no upward trend*. And this is the metric that matters. For scientific discovery you want **one breakthrough idea that dominates the baseline**, not a higher *average* over many safe ideas. RL improved the base rate and left the ceiling untouched — precisely backwards for discovery.

The cause — diversity collapse (Figures 6–7). Manual inspection shows the ideator converging onto a handful of easy-to-implement ideas that reliably earn positive reward. On nanoGPT it collapses to **two** ideas — (1) replace RMSNorm with LayerNorm, (2) EMA over checkpoints:



Simultaneously, **thinking traces shrink** over training (Figure 6) — the *opposite* of the emergent length-increase in DeepSeek-R1-style RLVR. The mechanism the authors identify: longer thinking → more complex ideas → **lower execution rate** → lower expected reward, so GRPO learns to think *less* and propose *simpler* ideas. The reward structure actively selects against the ambitious, hard-to-implement ideas that discovery requires. This is the open-ended cousin of pass@k collapse (Yue et al. 2025; Wu et al. "Invisible Leash", 2507.14843).

Attempted fixes (Appendix A.1), none decisive — dynamic prompt (append prior-epoch trajectories), a capped length reward (thinking stops shrinking but total reward doesn't rise), and a token-level Jaccard **similarity penalty** (maintains diversity but no clear gain over vanilla). All early-stopped. The authors are explicit that escaping collapse likely needs *new* algorithmic interventions beyond standard GRPO.

The two learners head-to-head

Dimension	Evolutionary search	RL from execution reward
what is updated	ideator PROMPT/context (frontier model, no weight upd.)	ideator WEIGHTS (GRPO gradient) (Qwen3-30B-A3B)
sample efficiency	HIGH – beats baselines & best-of-N within 10 epochs	LOW – 40-68 epochs, 256-1024 GPUs/step, only moves the mean
average idea quality	n/a (selects, doesn't shift the generator's distribution)	IMPROVES (0.253→0.343 acc; 0.194→0.246 reward)
maximum / upper-bound	IMPROVES – finds 69.4% / 3.1407; Opus shows a scaling trend	DOES NOT improve – max reward is flat / noisy
diversity	PRESERVED (explicit explore branch keeps the frontier wide)	COLLAPSES (converges to ~2 ideas; thinking length shrinks)
best suited to	DISCOVERY – finding the one breakthrough	raising the base RATE, not breakthroughs

The takeaway the paper draws: for *scientific discovery*, where you care about the max, the quality-diversity method (evolutionary search) is the right tool and vanilla GRPO is structurally mismatched — it optimizes the mean and pays for it with the tail.

Failure Modes

1. **Diversity / mode collapse in RL (the headline limitation).** Standard GRPO on an execution reward converges on a few trivial, easy-to-implement ideas, shrinks its reasoning, and never lifts the maximum. Monitor idea diversity and thinking length, not just mean reward; the preliminary length/similarity/dynamic-prompt fixes were not enough.
2. **Reward hacking.** An execution reward is only as good as its isolation. Two live attacks were observed and patched: future-token leakage via modified attention (fixed with a one-token-at-a-time validation function) and validation-metric tampering (fixed by isolating eval code the executor can't touch). The fixed-budget proxy-reward design (optimize 1/loss under a fixed 25-min budget, not wall-clock time directly) also removes a class of runtime-gaming exploits. Any new environment needs its own audit.
3. **Reproduction » discovery.** The search beats the *baseline it starts from* but stays $\sim 9.4\times$ slower than the human nanoGPT record (19.7 vs 2.1 min), and its "best" ideas often *rediscover* papers from the prior ~ 3 months rather than inventing new ones. This is progress on reproduction/reconstruction, not evidence of novel discovery.
4. **Executor capability ceiling → reward noise.** Ideas needing new packages, auxiliary models, or system-level changes fail to execute and score 0. Because failed execution = 0 reward, the whole loop is *biased toward easy-to-implement ideas* — a systematic pressure away from ambitious research, compounding the collapse in (1).
5. **Generalizability untested.** Best recipes are found at small scale (124M / 1.5B) under tight budgets; the paper does not test whether they transfer to larger models, other datasets, or longer runs — a stated limitation.

vs Alternatives

System	Optimizes	Reward / selection	Search operator	Scope	Key limitation
AI Scientist (Sakana) 2408.06292	a full paper pipeline	LLM reviewer score (the hackable proxy)	LLM ideation, no real exec loop	ML, open-ended	optimizes an idea-stage proxy; high failure rate
AlphaEvolve (DeepMind) 2506.13131	a single program / algorithm	PROGRAMMATIC evaluator (must be machine-checkable)	evolutionary over CODE diffs	any code with a machine-check metric	operates on code diffs, not NL ideas; needs a clean scalar evaluator
LLM Speedrunning Bench. (Meta) 2506.22419	REPRODUCE known nanoGPT records	wall-clock speedup vs a KNOWN target record	agent scaffold, per record (+hints)	nanoGPT records only	measures reproduction of known transitions, not open-ended discovery
THIS PAPER 2601.14525	a natural-language research IDEA	EXECUTION-grounded reward (real GPU expt)	(a) evolutionary search (b) GRPO from exec reward	pre- & post-training, open	RL mode-collapses; still reproduction » discovery; small-scale only

The clean positioning: AlphaEvolve is the closest relative and the search scaffold's inspiration, but it evolves *code* against a *given* evaluator; this paper evolves *natural-language ideas* (a higher level of abstraction) and adds the RL study. The Speedrunning Benchmark shares the nanoGPT substrate but scores *reproduction of known records*, whereas this paper scores *open-ended improvement*. AI Scientist is the thing being corrected — it is the generation pipeline whose idea-stage score the Ideation–Execution Gap invalidated.

Practical Takeaways

- **Ground idea selection in execution, not review scores.** This is the operational answer to the Ideation–Execution Gap: build the reward from run-it, not read-it. If you run an idea-generation loop, treat any LLM/reviewer idea-score as a proxy you should assume is being hacked, and gate on real outcomes.
- **For discovery, prefer quality-diversity search over vanilla RL.** If you want the *one* breakthrough (the max), use evolutionary search with an explicit exploration branch that protects diversity. GRPO raises the mean and collapses the tail — the wrong objective when the tail is the point. If you only need to raise a *base rate*, RL from execution reward works (0.253→0.343).

- **Budget for the reward, because the reward is an experiment.** One rollout's reward here is a distributed training job (256–1024 GPUs per step). Make it tractable with a proxy metric under a *fixed compute budget* (1/loss in 25 min, not raw wall-clock), which also kills runtime-gaming.
- **Isolate every reward channel.** Freeze and sandbox evaluation code; prevent future-token / label leakage. Execution rewards invite exactly these exploits.
- **Watch diversity and thinking length as first-class RL health metrics.** In open-ended RL, a rising mean reward with shrinking thinking traces is the *signature* of impending mode collapse — the same tail-collapse you'd monitor via pass@k in RLVR.
- **Two findings map straight onto your own runs.** The post-training win was "drop GRPO's clipping + importance weighting and tune HP" — a reminder that GRPO's extra machinery isn't always load-bearing at 1.5B/MATH scale. The pre-training win (EMA over checkpoints + learnable skip connections + wider SwiGLU + LR/WD tuning) is a concrete recipe to try on your GPT-2 124M speedrun.

Key Papers

1. Si, Yang, Choi, Candès, Yang, Hashimoto. *Towards Execution-Grounded Automated AI Research*. arXiv 2601.14525 (2026). <https://arxiv.org/abs/2601.14525> — this guide. Code: github.com/NoviScl/Automated-AI-Researcher.
2. Si, Hashimoto, Yang. *The Ideation–Execution Gap*. arXiv 2506.20803 (2025). <https://arxiv.org/abs/2506.20803> — the motivating negative result. See ideation-execution-gap-guide.md.
3. Si, Yang, Hashimoto. *Can LLMs Generate Novel Research Ideas?* ICLR 2025, arXiv 2409.04109 (2024). <https://arxiv.org/abs/2409.04109> — the "AI ideas are more novel" setup.
4. Novikov et al. *AlphaEvolve: A Coding Agent for Scientific and Algorithmic Discovery*. arXiv 2506.13131 (2025). <https://arxiv.org/abs/2506.13131> — search-scaffold inspiration.
5. *The Automated LLM Speedrunning Benchmark: Reproducing NanoGPT Improvements* (Meta). arXiv 2506.22419 (2025). <https://arxiv.org/abs/2506.22419> — the reproduction-focused sibling; forthcoming llm-speedrunning-benchmark-guide.md.
6. Jordan et al. *modded-nanogpt: Speedrunning the nanoGPT baseline*. 2024. <https://github.com/KellerJordan/modded-nanogpt> — the pre-training environment substrate.
7. Shao et al. *DeepSeekMath (GRPO)*. arXiv 2402.03300 (2024). <https://arxiv.org/abs/2402.03300> — the RL algorithm both your tutorial and this paper's RL variant use.
8. Wang et al. *RL for Reasoning with One Training Example*. NeurIPS 2025. <https://arxiv.org/abs/2504.20571> — the one-prompt RLVR setup the RL learner mirrors.
9. Yue et al. *Does RL Really Incentivize Reasoning Beyond the Base Model?* NeurIPS 2025; and Wu et al. *The Invisible Leash: Why RLVR May Not Escape Its Origin*. arXiv 2507.14843 (2025). <https://arxiv.org/abs/2507.14843> — the pass@k-collapse analogues of the diversity-collapse finding.
10. Lu et al. *The AI Scientist*. arXiv 2408.06292 (2024). <https://arxiv.org/abs/2408.06292> — the generation pipeline this line of work corrects. See ai-scientist-guide.md.

Verification note. All architecture details, both learners, the anti-reward-hacking measures, the two environments, and every headline number were transcribed directly from the arXiv 2601.14525 PDF (42 pages, read in full through the reference list and appendix): post-training 69.4% (Claude-4.5-Sonnet, epoch 2) vs 48.0% baseline vs 68.8% best human (CS336); pre-training 19.7 vs 35.9 min (Opus solution, val loss 3.1407 at epoch 9) vs 2.1 min human record; RL avg accuracy 0.253→0.343 (40 epochs), avg reward 0.194→0.246 / loss 5.150→4.066 (68 epochs), max flat; diversity collapse 51/128→119/128 (nanoGPT, two ideas); base ideator Qwen3-30B-A3B, group size 256/128 → 256/1024 GPUs; evolutionary batch 50/80, 10 epochs. Lineage arXiv IDs (2506.20803, 2409.04109, 2506.13131, 2506.22419, 2408.06292, and Wang et al.

one-training-example 2504.20571) were all confirmed by web lookup. No items are left [UNVERIFIED]; the only IDs sourced externally (rather than from the PDF body) are the cross-referenced lineage papers above, each confirmed against its arXiv landing page.*